

Adaptation de Domaine

Séance 6 — Fouille de Données Massives

Guillaume Metzler

Institut de Communication (ICOM), Université Lumière Lyon 2

Laboratoire ERIC UR 3083

guillaume.metzler@univ-lyon2.fr

M2 Informatique – SISE

Résumé

Que se passe-t-il quand le modèle qu'on a entraîné doit fonctionner sur des données qui ne ressemblent plus tout à fait à celles de l'entraînement? C'est la question de l'*adaptation de domaine* : on dispose de données *source* annotées, mais on veut être performant sur un domaine *cible* différent, pour lequel on n'a pas (ou peu) d'étiquettes. Ce document introduit le cadre formel de l'adaptation de domaine non supervisée, détaille en profondeur les méthodes de repondération d'instances (estimation de densité, KLIEP, KMM, classifieur de domaine), puis présente les bornes de généralisation qui expliquent *pourquoi* et *sous quelles conditions* ces méthodes fonctionnent (divergence $\mathcal{H}\Delta\mathcal{H}$, bornes PAC-Bayésiennes), avant d'ouvrir sur les approches profondes et récentes du domaine (DANN, transport optimal, test-time adaptation, modèles de fondation). Les bases sont volontairement traitées avec le même niveau de détail que le notebook `Code_DA.ipynb` du cours, pour que les notions les plus techniques restent solidement ancrées avant d'aborder les développements récents.

Table des matières

1	Introduction : pourquoi adapter un modèle ?	2
1.1	Trois exemples qui posent le problème	2
1.2	Domaine, tâche, risque	3
1.3	Pourquoi ne pas simplement réentraîner sur la cible?	4
1.4	Taxonomie du transfer learning	4
1.5	Transfert négatif et limites	4
1.6	Quatre hypothèses de correspondance source/cible	5
2	Adaptation de domaine non supervisée	6
2.1	Définition	6
2.2	Un exemple filé : régression 1D	6
2.3	Formalisation du problème	7
3	Approches instance-based	8
3.1	Le risque cible est un risque source repondéré	8
3.2	L'hypothèse de covariate shift simplifie tout	9

3.3	Formulation générale d'une approche instance-based	9
3.4	Panorama des méthodes d'estimation de $w(x)$	10
3.4.1	Estimation de densité paramétrique	10
3.4.2	Estimation de densité par noyaux (KDE)	11
3.4.3	Heuristique des plus proches voisins	11
3.4.4	Distribution matching : KLIEP et KMM	12
3.4.5	Classifieur de domaine (Importance Weighting Classifier)	13
3.5	Et quand le covariate shift ne tient pas, ou que les supports ne se recouvrent pas ?	14
4	Travaux récents : théorie et pratique	15
4.1	Pourquoi une borne de généralisation ?	15
4.2	La borne $\mathcal{H}\Delta\mathcal{H}$ de Ben-David et al.	15
4.3	Bornes PAC-Bayésiennes pour l'adaptation de domaine	17
4.4	Une mise en garde théorique récente : les limites du « domain-invariant » . . .	18
4.5	DANN : rendre les représentations invariantes par apprentissage adversarial .	18
4.6	Panorama pratique complémentaire	19
5	Conclusion	20
A	Réponses aux questions d'auto-évaluation	20

1 Introduction : pourquoi adapter un modèle ?

Ce premier chapitre pose le décor. Nous allons voir, sur trois exemples concrets, pourquoi un modèle entraîné dans de bonnes conditions peut se révéler décevant une fois déployé ailleurs ; nous donnerons ensuite un nom précis à ce qui change d'un contexte à l'autre — le *domaine* et la *tâche* — avant de situer l'adaptation de domaine dans la famille plus large du *transfer learning*, et de préciser les hypothèses sous lesquelles on peut raisonnablement espérer transférer une connaissance d'un domaine à un autre.

1.1 Trois exemples qui posent le problème

Le premier exemple vient de la reconnaissance d'objets : un système de vision entraîné avec les images d'une caméra particulière (éclairage, angle, résolution donnés) doit, une fois déployé, fonctionner avec une caméra différente, installée ailleurs. Les images « se ressemblent » conceptuellement — ce sont toujours des photographies des mêmes types d'objets — mais leurs statistiques (couleurs, contraste, cadrage) diffèrent sensiblement. Le deuxième exemple est linguistique : un classifieur entraîné sur des avis clients concernant des livres doit être réutilisé pour juger des avis sur de l'électroménager. Le vocabulaire change du tout au tout (« passionnant », « bien écrit » d'un côté, « silencieux », « durable » de l'autre), mais la tâche — détecter un avis positif ou négatif — reste rigoureusement la même. Le troisième exemple, enfin, est industriel : un modèle de détection de défauts est entraîné sur des données *simulées*, moins coûteuses à produire en grand nombre, puis doit fonctionner sur des mesures

réelles, dont la distribution ne coïncide jamais tout à fait avec celle de la simulation.

Ces trois situations, bien que très différentes en apparence, partagent la même structure : on dispose d'un domaine *source* où l'apprentissage est facile — les données y sont abondantes et étiquetées — et l'on souhaite être performant sur un domaine *cible*, différent, où les étiquettes sont rares, coûteuses à obtenir, voire totalement absentes. On retrouve ce schéma dans un nombre considérable de champs applicatifs : la détection d'anomalies, la biologie, la médecine, les jeux vidéo, la localisation intérieure ou encore le traitement du signal. Le *transfer learning* n'est donc pas une méthode isolée, mais bien une famille de problèmes qui traverse presque tout le machine learning appliqué.

1.2 Domaine, tâche, risque

Pour aller plus loin, il nous faut un langage précis. Nous reprenons pour cela le formalisme standard de l'apprentissage statistique, que nous étendons au cadre du transfert.

Définition 1.1: Domaine et tâche

Soit $\mathcal{X}$ un espace de features et $\mathcal{Y}$ un espace d'étiquettes, et (X, Y) un couple de variables aléatoires sur $\mathcal{X} \times \mathcal{Y}$ de loi P . On appelle **domaine** le couple $\mathcal{D} = \{\mathcal{X}, P(X)\}$ — l'espace des features et sa distribution marginale — et **tâche** le couple $\mathcal{T} = \{\mathcal{Y}, P(Y | X)\}$ — l'espace des étiquettes et la loi conditionnelle que l'on cherche à prédire.

Autrement dit, le domaine décrit « à quoi ressemblent les données d'entrée » à travers la distribution marginale $P(X)$, tandis que la tâche décrit « ce que l'on cherche à prédire » à travers la loi conditionnelle $P(Y | X)$. Un même jeu de données peut ainsi changer de domaine — les images changent de style — sans changer de tâche, puisqu'on cherche toujours à reconnaître un chat ; et réciproquement.

Définition 1.2: Transfer learning

On appelle **transfer learning** le cadre d'apprentissage qui exploite la connaissance acquise sur un domaine/tâche *source* $(\mathcal{D}_S, \mathcal{T}_S)$ pour améliorer l'apprentissage sur un domaine/tâche *cible* différent mais lié $(\mathcal{D}_T, \mathcal{T}_T)$. On parle de *décalage de domaine* (*domain shift*) si $\mathcal{X}_S \neq \mathcal{X}_T$ ou $P_S(X) \neq P_T(X)$, et de *décalage de tâche* (*task shift*) si $\mathcal{Y}_S \neq \mathcal{Y}_T$ ou $P_S(Y | X) \neq P_T(Y | X)$.

Pour comparer des modèles entre eux, il nous faut aussi rappeler la notion de risque, familière depuis le début du cours.

Définition 1.3: Risque d'un classifieur/régresseur

Pour une fonction de perte $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$ et une hypothèse $h : \mathcal{X} \rightarrow \mathcal{Y}$, le risque de h sous une distribution P est

$$R(h) = \mathbb{E}_{(X,Y) \sim P} [\ell(h(X), Y)].$$

En pratique, on ne connaît P qu'à travers un échantillon i.i.d. $\{(x_i, y_i)\}_{i=1}^N$, et on minimise le risque empirique $\hat{R}(h) = \frac{1}{N} \sum_{i=1}^N \ell(h(x_i), y_i)$ (principe ERM).

Muni de ces définitions, le point de départ de tout ce cours tient en une phrase, aussi

simple que fondamentale : dès que $P_S \neq P_T$, le risque source $R_S(h)$ et le risque cible $R_T(h)$ sont deux quantités *différentes*. Minimiser $\hat{R}_S(h)$, ce que l'on sait très bien faire, n'a donc *aucune raison a priori* de minimiser $R_T(h)$, ce que l'on veut vraiment faire. Tout l'enjeu de l'adaptation de domaine consiste précisément à relier ces deux risques l'un à l'autre.

1.3 Pourquoi ne pas simplement réentraîner sur la cible ?

On pourrait légitimement se demander pourquoi ne pas tout simplement oublier la source et entraîner un modèle directement sur la cible. En pratique, cette stratégie échoue le plus souvent, et ce pour plusieurs raisons qui se combinent souvent entre elles. D'abord, on dispose fréquemment de très peu de données cible : l'annotation peut être coûteuse, lente, voire dangereuse — pensons à la détection de pannes rares sur une infrastructure critique. Ensuite, dans le cadre qui nous intéresse ici, les données cible sont tout simplement *non annotées* : c'est même la définition du cadre non supervisé que nous allons étudier dans ce cours. Il arrive aussi que l'échantillon cible dont on dispose ne soit que *partiellement représentatif* du domaine cible réel, ou encore que certaines features présentes côté source manquent côté cible — un capteur disponible en laboratoire, mais absent sur le produit final, en est un exemple typique.

1.4 Taxonomie du transfer learning

Face à cette diversité de situations, il est utile de se doter d'une taxonomie. On distingue en général deux grandes façons de classer les approches de transfert : une classification *par le problème*, selon ce qui diffère entre source et cible, et une classification *par la solution*, selon le type de stratégie algorithmique employée.

Du point de vue du problème, on parle de **transfert hétérogène** lorsque l'espace des features ou celui des étiquettes change lui-même entre les deux domaines ($\mathcal{X}_S \neq \mathcal{X}_T$ ou $\mathcal{Y}_S \neq \mathcal{Y}_T$) — par exemple lorsqu'on transfère une tâche de reconnaissance d'objets depuis des images de profondeur vers des images classiques, ou lorsqu'on passe d'une tâche de détection d'instruments de musique à une tâche de reconnaissance de style musical. On distingue également les situations selon la disponibilité des étiquettes : le transfert est dit *inductif* lorsque des étiquettes sont disponibles à la fois côté source et côté cible, *transductif* lorsqu'elles ne le sont que côté source, et **entièrement non supervisé** lorsqu'aucune étiquette cible n'est disponible — c'est précisément le cadre de ce cours.

Du point de vue de la solution, trois grandes familles de méthodes se dégagent. Les approches **instance-based** répondent les instances source et cible pour corriger le décalage de distribution, avant d'entraîner un modèle sur l'ensemble ainsi repondéré : c'est le cœur de ce cours. Les approches **feature-based** cherchent, elles, une représentation commune qui gomme les différences entre domaines — nous les aborderons en ouverture, à la Section 4. Les approches **parameter-based**, enfin, partent des paramètres d'un modèle pré-entraîné sur la source et les adaptent à la cible, dans l'esprit du fine-tuning. Cette double taxonomie est reprise, entre autres, du survey de référence de [Zhuang et al. \(2021\)](#), qui reste une bonne carte pour se repérer dans une littérature très fragmentée.

1.5 Transfert négatif et limites

Il faut toutefois se garder de croire que le transfert est toujours bénéfique. On parle de **transfert négatif** lorsque la connaissance issue de la source *dégrade* en réalité la perfor-

mance sur la cible, par rapport à un apprentissage qui se limiterait aux (rares) données cible disponibles. Ce phénomène survient typiquement dans trois cas de figure : lorsque la dissimilarité entre domaines source et cible est trop importante, lorsque les tâches source et cible ne sont en fait pas suffisamment liées l'une à l'autre, ou lorsque la stratégie de transfert choisie n'est simplement pas adaptée au problème posé. Un réflexe utile à conserver en pratique consiste donc à toujours comparer une méthode d'adaptation de domaine à la baseline « source seule », et, si quelques étiquettes cible sont disponibles, à la baseline « cible seule ».

Le transfer learning est enfin à distinguer de domaines de recherche voisins mais différents, avec lesquels on le confond parfois : l'apprentissage multi-source, l'apprentissage multi-tâches, l'apprentissage par renforcement, ou encore le transport optimal, que nous retrouverons à la Section 4.

1.6 Quatre hypothèses de correspondance source/cible

Puisque $P_S \neq P_T$ en général, il est nécessaire de préciser *comment* les deux distributions sont liées l'une à l'autre pour espérer transférer quoi que ce soit de l'une vers l'autre : sans une telle hypothèse, rien ne permettrait de relier ce que l'on observe sur la source à ce qui se passe sur la cible. Quatre hypothèses structurent l'essentiel de la littérature.

Définition 1.4: Covariate shift

La distribution marginale change entre source et cible, mais la dépendance prédictive reste la même :

$$P_S(X) \neq P_T(X) \quad \text{mais} \quad P_S(Y | X) = P_T(Y | X).$$

Définition 1.5: Target shift

Les proportions de classes changent, mais pas la distribution des features à l'intérieur d'une classe :

$$\forall y \in \mathcal{Y}, \quad P_S(Y = y) \neq P_T(Y = y) \quad \text{mais} \quad P_S(X | Y = y) = P_T(X | Y = y).$$

Définition 1.6: Biais de sélection d'échantillon

Il existe une seule distribution sous-jacente P , mais une variable de sélection cachée ξ tend à exclure certaines observations de l'échantillon source :

$$P_S(X, Y) = \mathbb{P}(\xi = 1 | X, Y) P(Y | X) P(X), \quad P_T(X, Y) = P(X, Y).$$

Définition 1.7: Concept shift

Les deux lois conditionnelles changent : $P_T(Y | X) \neq P_S(Y | X)$ et $P_T(X | Y) \neq P_S(X | Y)$. C'est le cas le plus difficile : sans autre information, plus rien ne relie les deux domaines.

Dans ce cours, nous nous concentrerons presque exclusivement sur le **covariate shift**, qui est à la fois l'hypothèse la plus exploitée en pratique — sondages électoraux, capteurs industriels, pour ne citer que deux exemples déjà évoqués — et celle qui donne lieu aux méthodes de repondération les plus simples et les plus interprétables, comme nous allons le

voir dès le prochain chapitre.

Testez votre compréhension

Questions courtes, sans calcul ; réponses en Annexe A.

- Q1.1.** Un modèle entraîné sur des photos prises de jour doit fonctionner de nuit. Le domaine change-t-il, la tâche change-t-elle, ou les deux ?
- Q1.2.** Donnez un exemple de situation réelle relevant du *target shift* plutôt que du *covariate shift*.
- Q1.3.** Pourquoi le transfert négatif est-il plus probable quand $\mathcal{D}_S$ et $\mathcal{D}_T$ sont « trop différents » ?

2 Adaptation de domaine non supervisée

Fort de ce cadre général, nous pouvons maintenant nous concentrer sur le problème précis qui occupe le reste de ce cours : l'adaptation de domaine non supervisée. Nous en donnons d'abord une définition formelle, avant de dérouler un exemple filé de régression qui permettra, tout au long du chapitre suivant, de visualiser concrètement chaque nouvelle notion introduite.

2.1 Définition

Définition 2.1: Unsupervised Domain Adaptation (UDA)

On dispose d'un échantillon source **labellisé** $S = \{(x_1, y_1), \dots, (x_m, y_m)\}$, avec $x_i \sim P_S(X)$ et $y_i \sim P_S(Y \mid X = x_i)$, ainsi que d'un échantillon cible **non labellisé** $T_X = \{x'_1, \dots, x'_n\}$, avec $x'_j \sim P_T(X)$. L'objectif est d'apprendre une hypothèse h qui minimise le risque cible $R_T(h)$, *sans jamais observer d'étiquette cible*.

2.2 Un exemple filé : régression 1D

Pour fixer les idées, imaginons un praticien qui étudie la relation entre une variable X — l'âge, le poids d'un patient — et une variable clinique Y , par exemple l'évolution d'une maladie. Il dispose de trente patients *source* (x_i, y_i) pour ajuster un modèle linéaire $Y = aX + b$, et souhaite l'appliquer à une nouvelle population *cible*, pour laquelle il n'observe pour l'instant que X (les x'_j), pas encore Y . Nous reprendrons cet exemple à chaque nouvelle notion du chapitre, afin de la visualiser concrètement avant de la généraliser.

La Figure 1 illustre la situation de départ : la population source (bleu) et la population cible (rouge) ont des distributions de features clairement différentes.

Que se passe-t-il si l'on ignore ce décalage et que l'on ajuste un modèle sur la source uniquement ? La Figure 2 en donne la réponse : le modèle capture bien la relation $Y = |X|$ là où il dispose de données, c'est-à-dire autour de $x = -1$, mais cette relation, une fois extrapolée, se révèle biaisée dès qu'on l'évalue sur la zone où vit réellement la population cible, autour de $x = +1$.

Ce qu'il faut retenir de cet exemple, c'est qu'il ne s'agit en rien d'un problème d'algorithme ou de capacité du modèle : même un modèle « parfait » sur la source, au sens du

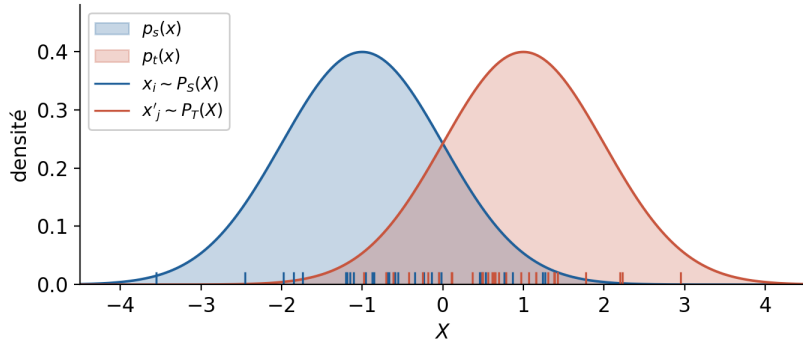


Figure 1 – Densités marginales $p_s(x)$ (source) et $p_t(x)$ (cible), avec les échantillons observés x_i et x'_j . Dans cet exemple, $P_S(X) = \mathcal{N}(-1, 1)$ et $P_T(X) = \mathcal{N}(1, 1)$.

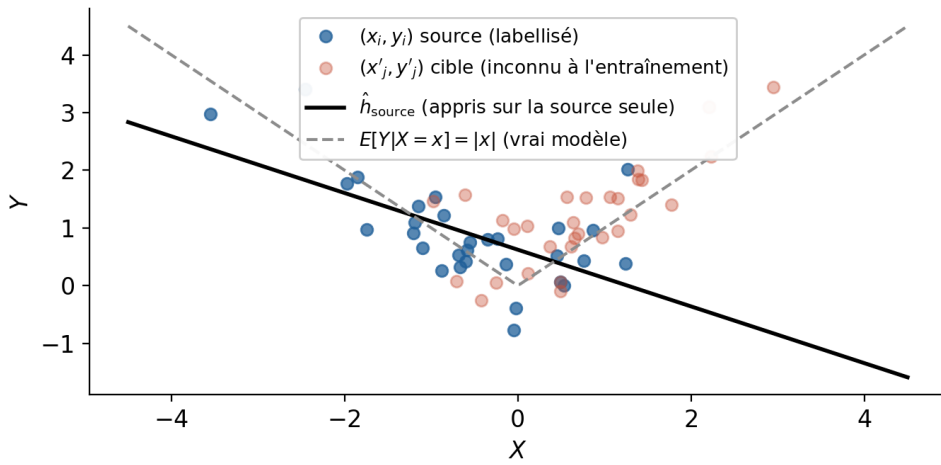


Figure 2 – Un modèle appris sur la source seule (noir) est correct près des observations source (bleu) mais s'écarte du vrai modèle cible $Y = |X|$ (gris pointillé) dans la zone où vit la cible (rouge).

principe ERM, peut se révéler arbitrairement mauvais sur la cible, dès lors que les deux distributions diffèrent et que la fonction cible n'est pas correctement extrapolée hors du support source. C'est très exactement le problème que l'adaptation de domaine cherche à résoudre.

2.3 Formalisation du problème

Formalisons à présent ce que nous venons d'observer. Notons $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$ la fonction de perte, par exemple $\ell(y, y') = \|y - y'\|_2^2$, et $\mathcal{H}$ un espace d'hypothèses — $\mathcal{H} = \{h : x \mapsto x\beta^\top, \beta \in \mathbb{R}^p\}$ pour des hypothèses linéaires. Le risque cible d'une hypothèse $h \in \mathcal{H}$ s'écrit alors

$$R_T(h) = \mathbb{E}_{(x,y) \sim P_T(X,Y)} [\ell(h(x), y)].$$

Si l'on disposait d'étiquettes cible y'_j , on l'estimerait naturellement par l'estimateur empirique $\hat{R}_T(h) = \frac{1}{n} \sum_{j=1}^n \ell(h(x'_j), y'_j)$ — mais ce n'est précisément pas le cas ici, puisque nous avons choisi de travailler dans le cadre non supervisé.

Définition 2.2: Problématique de l'UDA

L'hypothèse cible optimale est

$$h^* = \arg \min_{h \in \mathcal{H}} \mathbb{E}_{(x,y) \sim P_T(X,Y)} [\ell(h(x), y)].$$

Le but de l'UDA est de trouver la meilleure estimation possible de h^* à partir du seul échantillon source labellisé S et de l'échantillon cible non labellisé T_X .

C'est cette question — comment approcher h^* sans jamais voir d'étiquette cible — qui va occuper tout le chapitre suivant.

3 Approches instance-based

Nous allons voir dans ce chapitre que, sous l'hypothèse de covariate shift, le risque cible peut s'écrire comme un risque source *repondéré* : c'est l'identité fondatrice de toute la famille d'approches dites *instance-based*. Une fois cette identité établie, la question devient purement algorithmique — comment estimer les poids nécessaires? — et nous explorerons pour y répondre plusieurs familles de méthodes, de l'estimation de densité la plus naïve jusqu'au classifieur de domaine, qui nous servira de pont vers les méthodes profondes du chapitre suivant.

3.1 Le risque cible est un risque source repondéré

Proposition 3.1: Changement de mesure

Supposons que $P_S(X, Y)$ et $P_T(X, Y)$ admettent des densités $p_s(x, y)$, $p_t(x, y)$, et posons $w(x, y) = p_t(x, y)/p_s(x, y)$. Alors, pour toute hypothèse $h \in \mathcal{H}$,

$$\mathbb{E}_{(x,y) \sim P_T(X,Y)} [\ell(h(x), y)] = \mathbb{E}_{(x,y) \sim P_S(X,Y)} [w(x, y) \ell(h(x), y)].$$

Démonstration.

$$\mathbb{E}_{P_T} [\ell(h(x), y)] = \int p_t(x, y) \ell(h(x), y) dx dy = \int \frac{p_t(x, y)}{p_s(x, y)} p_s(x, y) \ell(h(x), y) dx dy = \mathbb{E}_{P_S} [w(x, y) \ell(h(x), y)].$$

□

Définition 3.2: Ratio de densités

Le **ratio de densités** (ou *importance weight*) est $w(x, y) = p_t(x, y)/p_s(x, y)$, défini pour (x, y) tel que $p_s(x, y) > 0$. On exige que le support de P_T soit inclus dans celui de P_S : $p_s(x, y) = 0 \Rightarrow p_t(x, y) = 0$, faute de quoi il existerait une zone cible jamais observée côté source, qu'aucune repondération ne pourrait corriger.

Cette identité est élégante, mais elle ne suffit pas encore, en l'état, à construire un algorithme implémentable : le ratio $w(x, y)$ fait intervenir $p_t(x, y) = p_t(y | x) p_t(x)$, et donc, *a priori*, les étiquettes cible — exactement ce qui nous manque. Il nous faut une hypothèse supplémentaire pour nous en affranchir.

3.2 L'hypothèse de covariate shift simplifie tout

Proposition 3.3: Ratio de densités sous covariate shift

Sous l'hypothèse de covariate shift ($P_T(Y | X = x) = P_S(Y | X = x)$ pour tout x), le ratio de densités ne dépend plus de y :

$$w(x, y) = w(x) = \frac{p_t(x)}{p_s(x)}.$$

Démonstration. $w(x, y) = \frac{p_t(x, y)}{p_s(x, y)} = \frac{p_t(y | x) p_t(x)}{p_s(y | x) p_s(x)} = \frac{p_t(x)}{p_s(x)}$ car $p_t(y | x) = p_s(y | x)$ par hypothèse. $\square$

C'est un progrès décisif : $w(x)$ ne dépend plus désormais que des *features*, que l'on observe des deux côtés, source et cible. On peut donc espérer l'estimer sans la moindre étiquette cible — ce qui était précisément l'obstacle rencontré à l'instant. Les sondages électoraux en offrent une bonne illustration : on y suppose que la population sondée vote comme elle répond au sondage, autrement dit que $P(\text{vote} | \text{profil})$ est identique entre l'échantillon interrogé et la population réelle. Les mesures industrielles en offrent une seconde : la machine de mesure utilisée pour construire le jeu d'entraînement est supposée avoir le même bruit de mesure que celle utilisée en production. Ce sont là deux illustrations réelles, dans des contextes très différents, de la même hypothèse de covariate shift.

Proposition 3.4: Risque cible sous covariate shift

Sous covariate shift, en supposant $w(x)$ connu pour tout x , le risque cible peut être estimé uniquement à partir de l'échantillon source labellisé :

$$R_T(h) = \mathbb{E}_{(x, y) \sim P_S(X, Y)} [w(x) \ell(h(x), y)] \approx \frac{1}{m} \sum_{i=1}^m w(x_i) \ell(h(x_i), y_i).$$

Récapitulons le chemin parcouru jusqu'ici. Un décalage de domaine implique que risque source et risque cible sont deux quantités distinctes ; sous covariate shift, le second s'écrit comme le premier repondéré par $w(x) = p_t(x)/p_s(x)$; et comme ce poids ne dépend que des features, il devient estimable sans la moindre étiquette cible. Reste, précisément, à l'estimer à partir des échantillons S_X et T_X dont on dispose réellement — c'est l'objet du reste de ce chapitre.

3.3 Formulation générale d'une approche instance-based

Définition 3.5: UDA instance-based

Une approche instance-based résout le problème d'optimisation en deux temps : on estime $\hat{w}(x_i)$ pour tout $x_i \in S_X$, à partir de S_X et T_X seuls, puis on résout

$$\hat{h}^* = \arg \min_{h \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \hat{w}(x_i) \ell(h(x_i), y_i).$$

Avant d'étudier comment estimer $\hat{w}$ en pratique, traitons l'exercice suivant, qui montre que la seconde étape — une fois les poids connus — se résout de façon entièrement explicite dans le cas d'une régression linéaire.

Exercice 3.1 (Régression pondérée — solution analytique). Soit $X \in \mathbb{R}^{m \times p}$ la matrice des observations, $Y \in \mathbb{R}^m$ le vecteur des cibles, $\ell(y, y') = (y - y')^2$, $\mathcal{H} = \{h : x \mapsto x\beta^\top, \beta \in \mathbb{R}^p\}$, et $w_i = p_t(x_i)/p_s(x_i)$ supposé connu. Montrez que la solution du problème repondéré $\beta^* = \arg \min_{\beta} \sum_{i=1}^m w_i (x_i \beta^\top - y_i)^2$ s'écrit, en notant $\Delta = \text{diag}(\sqrt{w_1}, \dots, \sqrt{w_m})$,

$$\beta^* = [X^\top \Delta^2 X]^{-1} X^\top \Delta^2 Y.$$

Solution. On écrit $\sum_i w_i (x_i \beta^\top - y_i)^2 = \|\Delta(X\beta^\top - Y)\|_2^2 = \|\Delta X\beta^\top - \Delta Y\|_2^2$. C'est un problème de moindres carrés ordinaires sur les données transformées ΔX , ΔY , dont la solution (en supposant $X^\top \Delta^2 X$ inversible) est $\beta^* = [(\Delta X)^\top \Delta X]^{-1} (\Delta X)^\top \Delta Y = [X^\top \Delta^2 X]^{-1} X^\top \Delta^2 Y$. $\square$

La Figure 3 illustre l'effet de cette repondération sur notre exemple filé : quand $w(x)$ est connu exactement, le modèle repondéré se rapproche nettement du vrai modèle cible, là où le modèle source seul en était très éloigné.

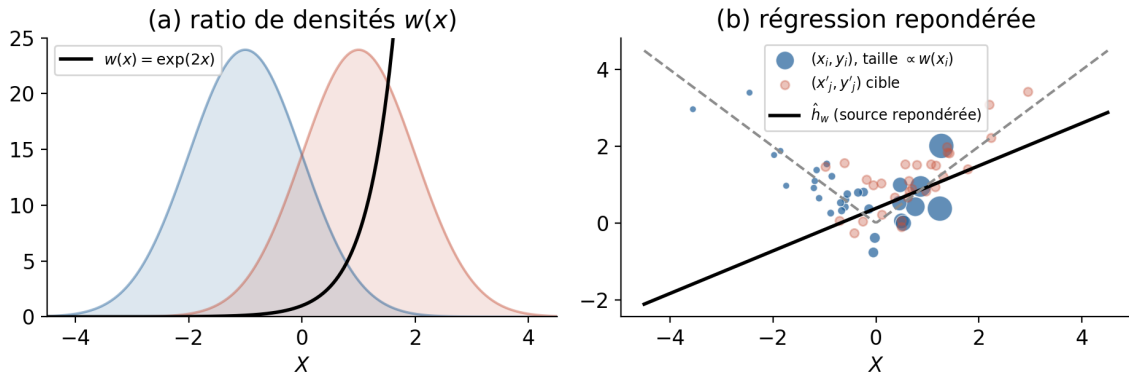


Figure 3 – (a) Ratio de densités exact $w(x)$ pour deux gaussiennes décalées de moyennes -1 et 1 . (b) Une fois les points source repondérés (taille $\propto w(x_i)$), le modèle appris ($\hat{h}_w$, noir) se rapproche du vrai modèle cible (gris pointillé), alors que le modèle non repondéré en était très éloigné (cf. Figure 2).

Il faut toutefois garder à l'esprit que, dans cet exemple, $w(x)$ a pu être calculé *analytiquement*, puisque nous connaissons par construction p_s et p_t — deux gaussiennes aux paramètres fixés à l'avance. En pratique, ces densités ne sont **jamais** connues : on n'observe que des échantillons S_X et T_X . La question devient donc : comment estimer $w(x)$ à partir de ces deux échantillons seuls ? C'est ce que nous allons voir maintenant, en parcourant les quatre grandes familles de méthodes résumées dans le Tableau 1.

3.4 Panorama des méthodes d'estimation de $w(x)$

3.4.1 Estimation de densité paramétrique

L'idée la plus directe consiste à supposer un modèle de densité $p_s(\theta_s, x)$, $p_t(\theta_t, x)$, par exemple gaussien, puis à estimer θ_s et θ_t par maximum de vraisemblance sur S_X et T_X respectivement, avant de poser $\hat{w}(x_i) = p_t(\hat{\theta}_t, x_i)/p_s(\hat{\theta}_s, x_i)$. Cette simplicité a toutefois un

Famille	Méthodes	Idée
Estimation de densité	Paramétrique ; noyaux (KDE)	Estimer $\hat{p}_s, \hat{p}_t$ séparément, puis $\hat{w} = \hat{p}_t/\hat{p}_s$
Heuristique	NN weighting (Loog, 2012)	Compter, pour chaque point source, combien de fois il est plus proche voisin d'un point cible
Distribution matching	KLIEP (Sugiyama et al., 2007), KMM (Huang et al., 2007)	Chercher directement w minimisant un écart entre $w \cdot p_s$ et p_t
Classifieur de domaine	Linéaire/noyau (Bickel et al., 2007), réseau de neurones (Zhang et al., 2018)	Relier $w(x)$ à la probabilité que x vienne de la cible plutôt que de la source

Table 1 – Panorama des approches d'estimation du ratio de densités.

coût : elle exige de *deviner* la bonne famille paramétrique. Si la vraie distribution n'est pas gaussienne — un mélange de plusieurs modes, une forte asymétrie — l'estimation du ratio se retrouve biaisée, parfois radicalement, comme le montre le notebook de TP sur une distribution exponentielle.

3.4.2 Estimation de densité par noyaux (KDE)

Pour s'affranchir du choix d'une famille paramétrique, on peut estimer les densités par noyaux : pour un noyau $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}_+$, on pose

$$\hat{p}_s(x) = C_s \sum_{x_i \in S_X} k(x, x_i), \quad \hat{p}_t(x) = C_t \sum_{x'_j \in T_X} k(x, x'_j),$$

puis $\hat{w}(x_i) = \hat{p}_t(x_i)/\hat{p}_s(x_i)$. Le noyau gaussien $k(x, x') = \exp(-\|x - x'\|_2^2/2\sigma^2)$ est le choix le plus courant, le paramètre σ , appelé bande passante, étant réglé par validation croisée sur la vraisemblance. Il reste que cette famille de méthodes — paramétrique ou à noyaux — estime les deux densités *séparément* avant d'en prendre le rapport, ce qui est un problème structurellement plus difficile que d'estimer directement le ratio lui-même : les erreurs des deux estimations se cumulent, en particulier dans les zones de faible densité source, où le ratio explose. C'est précisément cette observation qui motive les approches suivantes, qui cherchent toutes à estimer w *directement*, sans passer par l'étape intermédiaire des deux densités.

3.4.3 Heuristique des plus proches voisins

Définition 3.6: Nearest Neighbors Weighting

Pour $K \in \mathbb{N}^*$, notons $V_K(x'_j)$ l'ensemble des K plus proches voisins de $x'_j \in T_X$ dans S_X . Le poids de $x_i \in S_X$ est directement le nombre de fois où il apparaît comme voisin d'un point cible :

$$\hat{w}(x_i) = \text{Card}(\{x'_j \in T_X \mid x_i \in V_K(x'_j)\}).$$

L'intuition qui sous-tend cette heuristique, due à Loog (2012), est la suivante : un point source « entouré » de nombreux points cible est sous-représenté relativement à la cible, et doit donc voir son poids *augmenter*. À l'inverse, un point source isolé de toute observation

cible n'apporte presque rien à la tâche cible, et son poids doit rester proche de zéro. C'est une heuristique d'une grande simplicité, qui ne fait aucune hypothèse de forme sur les densités sous-jacentes, mais dont le comportement reste sensible au choix de K , comme l'illustre la Figure 4.

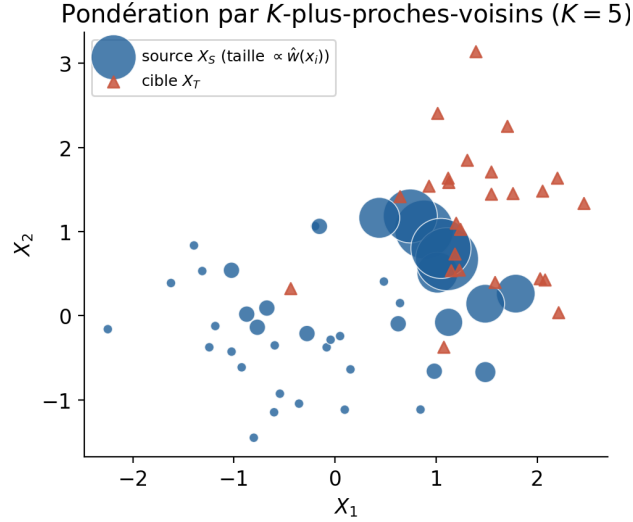


Figure 4 – Pondération par plus proches voisins en dimension 2 : les points source (bleu) qui sont souvent sélectionnés comme voisins de points cible (triangles rouges) reçoivent un poids plus élevé (taille du marqueur).

3.4.4 Distribution matching : KLIEP et KMM

L'idée commune à cette nouvelle famille de méthodes est de renoncer à estimer p_s et p_t séparément, pour chercher *directement* le poids w qui rend la source repondérée $w \cdot p_s$ aussi proche que possible de la cible p_t , au sens d'une métrique de proximité D entre distributions :

$$w^* = \arg \min_w D(p_t(x), w(x) p_s(x)).$$

Deux choix de métrique D donnent lieu à deux algorithmes classiques, tous deux encore très employés aujourd'hui.

Définition 3.7: KLIEP

KLIEP (*Kullback–Leibler Importance Estimation Procedure*, Sugiyama et al., 2007) choisit $D = \text{KL}$. En écrivant w comme combinaison linéaire de noyaux $w(x) = \sum_{j=1}^b \alpha_j k(x, x'_j)$, le problème devient

$$\alpha^* = \arg \min_{\alpha \in \mathbb{R}^b} \sum_{x'_i \in T_X} \log \left(\sum_{j=1}^b \alpha_j k(x'_i, x'_j) \right) \quad \text{s.c.} \quad \sum_{x_i \in S_X} \sum_{j=1}^b \alpha_j k(x_i, x'_j) = 1, \quad \alpha \geq 0.$$

Définition 3.8: Kernel Mean Matching (KMM)

KMM (Huang et al., 2007) choisit $D = \text{MMD}$ (*Maximum Mean Discrepancy*), qui compare deux distributions à travers leurs moments dans un espace de Hilbert à noyau reproduisant. Le problème d'optimisation devient quadratique :

$$w^* = \arg \min_{w \in \mathbb{R}^m, w \geq 0, w^\top \mathbf{1} = m} w^\top K w - 2\kappa^\top w,$$

où $K_{ij} = k(x_i, x_j)$ pour $x_i, x_j \in S_X$ et $\kappa_i = \frac{m}{n} \sum_{x'_j \in T_X} k(x_i, x'_j)$.

La MMD compare deux distributions à travers leurs moments : $\mathbb{E}_{P_S}[X^k]$ contre $\mathbb{E}_{P_T}[X^k]$ pour tout ordre k , l'astuce du noyau permettant de faire ce calcul sans jamais expliciter des moments d'ordre élevé. Or deux distributions ont tous leurs moments égaux si, et seulement si, elles sont identiques : c'est très exactement ce principe que KMM exploite pour forcer $w \cdot p_s$ à « ressembler » à p_t en tout point, sans jamais avoir à estimer p_s ni p_t séparément. Son principal avantage pratique est de ne requérir que la résolution d'un programme quadratique convexe, sans aucune sélection de modèle de densité au préalable.

3.4.5 Classifieur de domaine (Importance Weighting Classifier)

Nous arrivons à la dernière famille de méthodes de ce chapitre, et sans doute la plus féconde des quatre : c'est elle qui inspirera directement les méthodes d'adaptation de domaine profondes du chapitre suivant.

Définition 3.9: Importance Weighting Classifier

On entraîne un classifieur $\phi : \mathcal{X} \rightarrow [0, 1]$ à discriminer les échantillons source (label = 0) des échantillons cible (label = 1) :

$$\phi^* = \arg \max_{\phi \in \Phi} \mathbb{E}_{P_S(X)} [\log \phi_0(X)] + \mathbb{E}_{P_T(X)} [\log \phi_1(X)]$$

(formulation par entropie croisée), puis on relie le ratio de densités à la sortie de ce classifieur :

$$w(x) = \frac{1}{\phi(x)} - 1, \quad \text{où } \phi(x) = \mathbb{P}(\text{domaine} = T \mid X = x).$$

L'idée sous-jacente à l'IWC est remarquablement simple, et mérite qu'on la démontre proprement : le ratio de densités $w(x)$ est directement relié à la probabilité qu'un point x appartienne au domaine cible plutôt qu'au domaine source, telle qu'estimée par un classifieur entraîné à discriminer les deux.

Exercice 3.2 (Justification de l'IWC). Soient $p_s(x)$ et $p_t(x)$ telles que $\text{supp}(p_t) \subset \text{supp}(p_s)$, et soit ϕ^* le classifieur de domaine optimal pour la perte d'entropie croisée :

$$\phi^* = \arg \max_{\phi} \mathbb{E}_{P_S(X)} [\log \phi(X)] + \mathbb{E}_{P_T(X)} [\log(1 - \phi(X))].$$

Montrez que $w(x) = p_t(x)/p_s(x) = 1/\phi^*(x) - 1$.

Solution. En développant les espérances en intégrales,

$$\phi^* = \arg \max_{\phi} \int_{\mathcal{X}} (p_s(x) \log \phi(x) + p_t(x) \log(1 - \phi(x))) dx.$$

Pour chaque x fixé, on maximise point par point $g(\phi) = p_s(x) \log \phi + p_t(x) \log(1 - \phi)$ en $\phi \in (0, 1) : g'(\phi) = p_s(x)/\phi - p_t(x)/(1 - \phi) = 0 \iff \phi^*(x) = \frac{p_s(x)}{p_s(x) + p_t(x)}$. On vérifie $g'' < 0$ (maximum). On en déduit

$$\frac{1}{\phi^*(x)} - 1 = \frac{p_s(x) + p_t(x)}{p_s(x)} - 1 = \frac{p_t(x)}{p_s(x)} = w(x). \quad \square$$

Une mise en garde s'impose ici, car elle donne lieu à un piège classique en TP : dans cet exercice, $\phi(x)$ est défini comme $\mathbb{P}(\text{domaine} = S \mid x)$ et non comme $\mathbb{P}(\text{domaine} = T \mid x)$, afin de retrouver exactement la formule du cours de référence sur lequel s'appuie cet exercice. Selon la convention choisie pour l'étiquetage 0/1 des domaines source et cible, le poids s'écrit donc soit $w(x) = 1/\phi(x) - 1$, soit $w(x) = \phi(x)/(1 - \phi(x))$: il faut **toujours** vérifier la convention en vigueur avant d'implémenter quoi que ce soit, sous peine d'inverser purement et simplement le sens de la repondération.

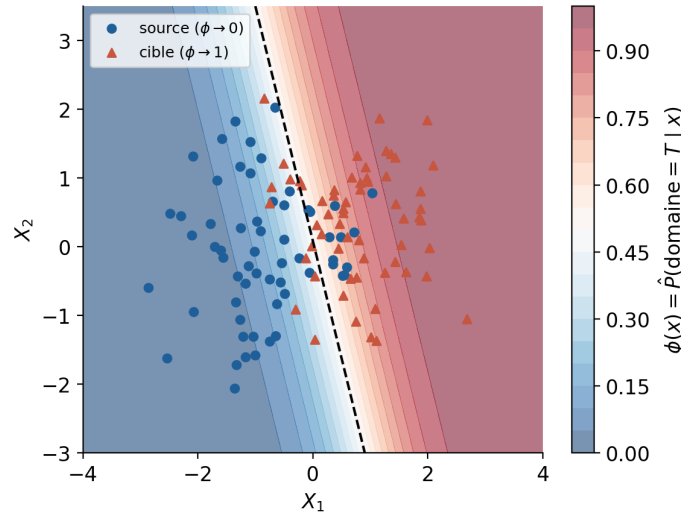


Figure 5 – Un classifieur de domaine $\phi(x) = \hat{\mathbb{P}}(T \mid x)$ (ici une régression logistique) sépare les points source (bleu, $\phi \rightarrow 0$) des points cible (rouge, $\phi \rightarrow 1$). Le ratio de densités $w(x) = \phi(x)/(1 - \phi(x))$ (convention $0 = S, 1 = T$) explose quand $\phi(x) \rightarrow 1$, c'est-à-dire dans les zones « typiquement cible ».

Le choix de la classe de classifieurs Φ pilote directement la complexité de la repondération obtenue : un classifieur linéaire produira un w « simple », correspondant à une frontière linéaire entre les deux nuages de points, tandis qu'un réseau de neurones produira un w beaucoup plus flexible — au risque, si le classifieur devient trop expressif par rapport à la taille des échantillons dont on dispose, de surapprendre et de produire des poids extrêmes concentrés sur quelques points isolés seulement.

3.5 Et quand le covariate shift ne tient pas, ou que les supports ne se recouvrent pas ?

Il convient de conclure ce chapitre par deux limites structurelles, communes à l'ensemble des méthodes que nous venons de voir. La première est logique : si $P_S(Y \mid X) \neq P_T(Y \mid X)$,

c'est-à-dire si l'hypothèse de covariate shift ne tient plus, repondérer les instances ne corrige absolument rien, car on ne fait alors que repondérer la distribution des X , sans toucher à la relation $X \rightarrow Y$ elle-même, qui a pourtant changé. La seconde est numérique : si $p_s(x) \approx 0$ dans une région où $p_t(x) > 0$, le ratio $w(x) = p_t(x)/p_s(x)$ explose, quelques points source reçoivent alors un poids disproportionné, et l'estimateur repondéré devient très instable, au sens d'une variance élevée. C'est de loin le symptôme le plus fréquent d'échec pratique des méthodes instance-based, et nous y reviendrons de façon quantitative au TD, à travers la notion d'*Effective Sample Size*.

C'est précisément pour répondre à ces deux limites que la littérature s'est progressivement tournée vers les approches *feature-based* : plutôt que de repondérer les points existants, on cherche un espace de représentation commun dans lequel les deux domaines se recouvrent davantage. C'est l'un des fils conducteurs du chapitre suivant.

4 Travaux récents : théorie et pratique

Ce dernier chapitre répond à deux questions restées jusqu'ici en suspens. La première est théorique : *pourquoi* les méthodes instance-based — et leurs cousines feature-based — fonctionnent-elles réellement, et sous quelles conditions précises peut-on garantir qu'elles amélioreront le risque cible ? Nous y répondrons à l'aide des bornes de généralisation qui relient formellement risque source et risque cible. La seconde question est pratique : quelles sont, aujourd'hui, les grandes familles d'algorithmes qui dominent l'adaptation de domaine en dehors du cadre instance-based que nous avons étudié en détail ? Nous parcourrons pour cela un panorama volontairement large, des réseaux adversariaux au transport optimal, en passant par l'adaptation au moment même du test.

4.1 Pourquoi une borne de généralisation ?

Jusqu'ici, nous disposons d'une méthode — repondérer — mais d'aucune garantie théorique : rien ne nous dit, pour une paire de domaines donnée, à quel point on peut espérer que $R_T(h)$ se rapproche de $\hat{R}_S(h)$, repondéré ou non. Les bornes de généralisation en adaptation de domaine répondent précisément à cette question, en général sous une forme structurée en trois termes :

$$R_T(h) \leq R_S(h) + (\text{terme de divergence entre domaines}) + (\text{terme irréductible}).$$

Les deux sections suivantes détaillent deux instances célèbres de cette forme générale : la borne de Ben-David et al., historiquement la première, puis son pendant PAC-Bayésien, souvent plus fin en pratique.

4.2 La borne $\mathcal{H}\Delta\mathcal{H}$ de Ben-David et al.

Définition 4.1: $\mathcal{H}$ -divergence

Pour une classe d'hypothèses $\mathcal{H}$ vue comme classe de classifieurs binaires $\mathcal{X} \rightarrow \{0, 1\}$, la **$\mathcal{H}$ -divergence** entre deux distributions marginales $D_S = P_S(X)$ et $D_T = P_T(X)$ est

$$d_{\mathcal{H}}(D_S, D_T) = 2 \sup_{h \in \mathcal{H}} \left| \mathbb{P}_{x \sim D_S}[h(x) = 1] - \mathbb{P}_{x \sim D_T}[h(x) = 1] \right|.$$

Cette quantité mesure à quel point le *meilleur* classifieur de la famille $\mathcal{H}$ parvient à distinguer un point source d'un point cible. Si $D_S = D_T$, aucun classifieur ne peut faire mieux que le hasard, et $d_{\mathcal{H}} = 0$; plus les domaines sont « séparables », plus $d_{\mathcal{H}}$ se rapproche de son maximum, égal à 2. On retrouve ici, sous un jour nouveau, exactement l'idée du classifieur de domaine introduit au chapitre précédent : entraîner un classifieur source-vs-cible n'était donc pas qu'une astuce pour estimer $w(x)$, c'est littéralement une façon d'estimer empiriquement $d_{\mathcal{H}}$.

Pour formuler une borne exploitable sur une classe d'hypothèses $\mathcal{H}$ donnée — et pas seulement sur des classifieurs binaires — il faut introduire une variante légèrement plus subtile de cette divergence.

Définition 4.2: Divergence $\mathcal{H}\Delta\mathcal{H}$

On introduit la classe symétrisée $\mathcal{H}\Delta\mathcal{H} = \{h \oplus h' : h, h' \in \mathcal{H}\}$, c'est-à-dire le XOR de deux hypothèses de $\mathcal{H}$ — l'ensemble des régions de désaccord entre deux hypothèses de $\mathcal{H}$ —, et

$$d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) = 2 \sup_{h, h' \in \mathcal{H}} \left| \mathbb{P}_{D_S}[h(x) \neq h'(x)] - \mathbb{P}_{D_T}[h(x) \neq h'(x)] \right|.$$

Nous disposons à présent de tous les ingrédients pour énoncer le résultat central de **Ben-David et al. (2010)**.

Théorème 4.3: Borne de Ben-David et al. (2010)

Soit $\mathcal{H}$ une classe d'hypothèses de classification, $R_S(h)$, $R_T(h)$ les risques (en 0/1-loss) sous les distributions jointes source et cible. Notons

$$\lambda_{\mathcal{H}} = \min_{h \in \mathcal{H}} (R_S(h) + R_T(h))$$

le risque joint optimal atteignable (irréductible : c'est le meilleur compromis possible entre les deux domaines). Alors, pour tout $h \in \mathcal{H}$,

$$R_T(h) \leq R_S(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(D_S, D_T) + \lambda_{\mathcal{H}}.$$

Idée de preuve. On ne reprend pas ici la démonstration complète, disponible dans **Ben-David et al. (2010)**, mais seulement son idée directrice. On introduit l'hypothèse jointe optimale $h^* = \arg \min_h (R_S(h) + R_T(h))$, et on décompose $R_T(h) \leq R_T(h^*) + \mathbb{P}_{D_T}[h \neq h^*]$ par l'inégalité triangulaire pour le désaccord entre classifieurs. On répète le même argument côté source, puis on relie $\mathbb{P}_{D_T}[h \neq h^*]$ et $\mathbb{P}_{D_S}[h \neq h^*]$ via la définition de $d_{\mathcal{H}\Delta\mathcal{H}}$, vue comme borne *sup* sur les désaccords, ce qui fait apparaître le terme de divergence recherché. $\square$

Cette borne dit quelque chose de très concret : minimiser le risque cible revient à minimiser simultanément, d'une part, le risque source, et d'autre part la divergence entre les deux domaines, mesurée dans l'espace de représentation utilisé. Le terme $\lambda_{\mathcal{H}}$ rappelle, quant à lui, qu'aucune méthode ne peut faire mieux que le meilleur compromis atteignable par la classe $\mathcal{H}$: si les tâches sont fondamentalement incompatibles, dans un cas extrême de concept shift par exemple, aucune astuce algorithmique ne saurait sauver la situation.

Il faut enfin noter que la $\mathcal{H}\Delta\mathcal{H}$ -divergence ne peut, en toute rigueur, pas être calculée exactement, puisqu'elle porte sur un supremum sur une classe de fonctions en général infinie.

Elle admet toutefois un proxy empirique parfaitement calculable : on entraîne un classifieur ϕ à discriminer S_X de T_X — exactement le classifieur de domaine du chapitre précédent — et l'on estime

$$\hat{d}_{\mathcal{H}}(S_X, T_X) = 2 \left(1 - 2 \min_{\phi \in \mathcal{H}} [\text{err}(\phi)] \right),$$

où $\text{err}(\phi) \in [0, \frac{1}{2}]$ désigne le taux d'erreur du *meilleur* classifieur de domaine de $\mathcal{H}$ à séparer source et cible : un classifieur incapable de faire mieux que le hasard a $\text{err}(\phi) = 1/2$, ce qui redonne bien $\hat{d}_{\mathcal{H}} = 0$, tandis qu'un classifieur parfait a $\text{err}(\phi) = 0$, ce qui redonne $\hat{d}_{\mathcal{H}} = 2$, la valeur maximale. C'est la **proxy $\mathcal{A}$ -distance** de Ben-David et al. (2010). Un classifieur de domaine qui se trompe beaucoup, proche du hasard, signale ainsi des domaines proches, tandis qu'un classifieur qui sépare parfaitement signale des domaines très éloignés — exactement l'intuition que suggérait déjà la Figure 5.

4.3 Bornes PAC-Bayésiennes pour l'adaptation de domaine

Les bornes de type Ben-David portent sur un unique classifieur h . L'approche PAC-Bayésienne, elle, raisonne sur une *distribution* Q de classifieurs — un vote de majorité — et permet d'obtenir des bornes souvent plus fines en pratique.

Définition 4.4: Cadre PAC-Bayésien pour le DA (Germain et al., 2013)

Soit $\mathcal{H}$ une classe d'hypothèses, Q une distribution (*posterior*) sur $\mathcal{H}$, et P un *prior* indépendant des données. On définit le risque de Gibbs sous Q , $R_{\mathcal{D}}(Q) = \mathbb{E}_{h \sim Q}[R_{\mathcal{D}}(h)]$ pour $\mathcal{D} \in \{S, T\}$, et le **désaccord de domaine** sous Q :

$$\text{dis}_T(Q) = \mathbb{E}_{(h, h') \sim Q^2} [\mathbb{P}_{x \sim D_T} [h(x) \neq h'(x)]] .$$

Théorème 4.5: Borne PAC-Bayésienne pour l'adaptation de domaine (Germain et al., 2013, 2016)

Sous des hypothèses techniques usuelles (perte bornée, échantillons i.i.d.), avec probabilité au moins $1 - \delta$ sur le tirage de S et T_X , pour tout posterior Q sur $\mathcal{H}$:

$$R_T(Q) \leq R_S(Q) + \frac{1}{2} \text{dis}_T(Q) + \lambda(Q) + O \left(\sqrt{\frac{\text{KL}(Q \| P) + \log \frac{1}{\delta}}{m}} \right),$$

où $\lambda(Q)$ est, comme précédemment, un terme irréductible dépendant du meilleur compromis source/cible atteignable sous Q , et $\text{KL}(Q \| P)$ est la divergence de Kullback-Leibler entre le posterior appris et le prior.

La structure de cette borne est exactement la même que celle de Ben-David — risque source, plus divergence de domaine, plus terme irréductible — mais deux différences comptent en pratique. D'une part, le désaccord $\text{dis}_T(Q)$ peut non seulement être estimé sans étiquette cible, mais aussi être optimisé directement en apprenant Q , plutôt que de se contenter d'un simple proxy de classifieur de domaine. D'autre part, le terme de complexité $\text{KL}(Q \| P)$ contrôle explicitement le compromis biais/variance : un posterior Q trop éloigné du prior P , donc très « spécialisé », dégrade la garantie théorique même s'il minimise parfaitement les deux premiers termes. C'est ce même principe qui sous-tend les certificats PAC-Bayésiens pour les méthodes d'ensemble étudiées ailleurs dans ce master. Ces bornes ont depuis été

raffinées par [Germain et al. \(2016\)](#), qui en propose des versions à la fois plus serrées et calculables pour des posteriors linéaires, utiles par exemple pour des SVM ou des modèles linéaires adaptés au domaine ; plus récemment encore, dans les années 2020, une ligne de travaux combine ce cadre avec le transport optimal — que nous verrons plus bas — pour obtenir des bornes exploitant directement une distance de Wasserstein entre domaines, souvent plus facile à estimer en haute dimension qu’une divergence de nature combinatoire.

4.4 Une mise en garde théorique récente : les limites du « domain-invariant »

Travaux récents

Une intuition très répandue depuis Ben-David et al. veut que, si l’on apprend une représentation où $\hat{d}_{\mathcal{H}}(S, T) \approx 0$ — des domaines devenus indistinguables — alors un bon transfert soit garanti. [Zhao et al. \(2019\)](#) montrent que cette intuition est **fausse en général sous target shift** : rendre les représentations totalement invariantes au domaine peut en réalité *augmenter* l’erreur jointe minimale $\lambda_{\mathcal{H}}$ elle-même, en détruisant l’information nécessaire pour distinguer les classes entre elles. Les auteurs établissent même des bornes *inférieures*, c’est-à-dire des résultats d’impossibilité, dès lors que les proportions de classes diffèrent trop entre source et cible. C’est un rappel salutaire : la borne de Ben-David constitue une condition *suffisante* pour contrôler un terme du risque, non une recette universelle — et ce résultat explique en partie pourquoi le simple fait de « tromper un classifieur de domaine », comme le fait DANN que nous introduisons ci-dessous, échoue parfois en présence d’un target shift marqué.

4.5 DANN : rendre les représentations invariantes par apprentissage adversarial

Travaux récents

[Ganin and Lempitsky \(2015\)](#) proposent une opérationnalisation directe de la borne de Ben-David : au lieu de repondérer des instances dans l’espace des features d’origine, on apprend un **extracteur de features** G_f , un réseau de neurones, tel qu’un classifieur de labels G_y appliqué sur $G_f(x)$ minimise le risque source comme d’habitude, tandis qu’un classifieur de domaine G_d appliqué sur $G_f(x)$ ne parvient plus, lui, à distinguer source et cible — ce qui, par construction, revient exactement à minimiser le proxy empirique de $d_{\mathcal{H}\Delta\mathcal{H}}$ introduit plus haut. L’astuce d’implémentation qui rend cela possible, le **Gradient Reversal Layer**, permet d’entraîner les trois morceaux du réseau (G_f , G_y , G_d) en une seule et même descente de gradient : le gradient de la perte de G_d est simplement *inversé* au moment où il traverse G_f lors de la rétropropagation, ce qui revient à faire, du point de vue de G_f , un pas de *maximisation* de l’erreur du classifieur de domaine, tout en continuant, du point de vue de G_d lui-même, une descente de gradient tout à fait classique. On obtient ainsi un jeu à deux joueurs, de type minimax, sans jamais avoir à implémenter de double boucle d’optimisation.

DANN peut ainsi se lire comme l’IWC du chapitre précédent, mais employé pour apprendre une représentation plutôt que pour repondérer des instances déjà existantes : le même classifieur de domaine ϕ , la même intuition selon laquelle un bon transfert rend source et cible indiscernables, mais une optimisation qui porte désormais sur les *features* elles-mêmes

plutôt que sur de simples poids scalaires. Cette extension permet de traiter des situations où aucune repondération d’instances ne suffirait — par exemple lorsque les supports sont initialement disjoints — au prix, toutefois, de la mise en garde énoncée à l’instant.

4.6 Panorama pratique complémentaire

Au-delà de DANN, plusieurs autres directions structurent aujourd’hui la pratique de l’adaptation de domaine, chacune répondant à une limite différente des méthodes vues dans ce cours.

Travaux récents

Alignement de moments (Deep CORAL, MMD-based deep DA). Plutôt qu’un jeu adversarial, on peut ajouter directement à la perte d’entraînement un terme qui pénalise l’écart entre les statistiques des features source et cible — la covariance pour CORAL (Sun and Saenko, 2016), la MMD pour DAN (Long et al., 2015) — ce qui constitue une extension directe de KMM à l’espace des features appris plutôt qu’à l’espace d’entrée brut.

Transport optimal pour l’adaptation de domaine. Courty et al. (2017) formulent l’adaptation de domaine comme un problème de *transport optimal* : on cherche le plan de transport γ le moins coûteux qui déplace la masse de p_s vers p_t , avant d’apprendre conjointement le classifieur et ce plan de transport (JDOT). La distance de Wasserstein qui en résulte a l’avantage d’une interprétation géométrique directe — elle mesure un « coût de déplacement » plutôt qu’un simple rapport de densités — et se comporte souvent mieux en haute dimension que KL ou MMD lorsque les supports se recouvrent peu, exactement la limite pointée en fin de Section 3.

Auto-apprentissage et pseudo-labellisation. Une fois un premier modèle adapté disponible, rien n’empêche de lui faire prédire des *pseudo-étiquettes* sur la cible, de ne garder que les prédictions les plus confiantes, puis de réentraîner en semi-supervisé sur la source et la cible ainsi pseudo-labellisée. C’est un principe d’une grande simplicité, itéré et raffiné (seuils adaptatifs par classe, par exemple), redevenu très compétitif sur les benchmarks de vision récents.

Test-Time Adaptation (TENT). Wang et al. (2021) posent un cadre encore plus contraint : au moment du test, on ne dispose **ni des données source, ni d’aucune étiquette**, seulement d’un flux de données cible non labellisées et d’un modèle déjà entraîné. TENT adapte alors uniquement les paramètres des couches de normalisation (*batch normalization*) du réseau, en minimisant l’entropie des prédictions sur les lots cible observés au fil du temps — une adaptation « à la volée », sans jamais revoir la source.

Adaptation de domaine avec des modèles de fondation. Depuis 2022, les modèles pré-entraînés à grande échelle, CLIP et ses variantes en tête, changent sensiblement la donne : leur espace de représentation, appris sur des milliards de paires image-texte, se révèle déjà remarquablement robuste aux décalages de domaine visuels usuels. Les travaux les plus récents, entre 2023 et 2025, portent ainsi moins sur l’alignement explicite de distributions que sur le *prompt tuning* ou l’ajout de petits adaptateurs légers, pour spécialiser ce modèle générique à la tâche cible avec très peu, voire aucune, donnée cible étiquetée — une forme de transfert où le « domaine source » est devenu si large et si divers que le décalage résiduel vers presque n’importe quelle cible réaliste devient gérable par un ajustement minimal.

Retenons de ce chapitre l'essentiel. La borne de Ben-David formalise le compromis entre risque source, divergence entre domaines, et risque joint irréductible, un socle théorique commun à presque toute la littérature de l'adaptation de domaine. Le classifieur de domaine n'y est d'ailleurs pas qu'un outil pratique : c'est un véritable estimateur de la $\mathcal{H}\Delta\mathcal{H}$ -divergence, ce qui explique directement pourquoi une méthode comme DANN fonctionne. Les bornes PAC-Bayésiennes généralisent ce cadre à des votes de majorité, avec un contrôle explicite de la complexité via $KL(Q\|P)$; mais rendre les représentations « invariantes au domaine » n'est en rien une garantie universelle, puisque sous target shift marqué, cela peut au contraire se révéler contre-productif, comme l'ont montré Zhao et al. en 2019. Le transport optimal, la pseudo-labellisation, le test-time adaptation et les modèles de fondation constituent enfin quatre directions actuelles particulièrement actives, chacune apportant sa propre réponse à une limite spécifique des méthodes étudiées dans ce cours.

5 Conclusion

L'adaptation de domaine part d'un constat simple — un modèle entraîné sur une distribution P_S n'a aucune garantie de bien fonctionner sous une autre distribution P_T — et propose une famille d'outils pour restaurer cette garantie, sous des hypothèses explicites sur la relation entre P_S et P_T . Les approches instance-based, sous l'hypothèse de covariate shift, offrent la voie la plus directe et la plus interprétable : repondérer les exemples source pour que leur distribution repondérée « ressemble » à la cible. Le principal défi pratique reste l'estimation de ce poids, de la simple estimation de densité au classifieur de domaine, en passant par le distribution matching. Les bornes de Ben-David et les bornes PAC-Bayésiennes expliquent pourquoi ces méthodes marchent, et surtout quand elles risquent d'échouer, et se retrouvent, décennie après décennie, au cœur des méthodes profondes les plus récentes — DANN, transport optimal, test-time adaptation. Retenez surtout ceci : avant d'appliquer n'importe quelle méthode d'adaptation de domaine, demandez-vous quelle hypothèse de correspondance source/cible, parmi celles de la Section 1.6, est plausible pour votre problème — car c'est elle qui déterminera, en dernier ressort, si la méthode a une chance de fonctionner.

A Réponses aux questions d'auto-évaluation

- Q1.1 Le domaine change (l'apparence des images, donc $P(X)$) ; si la tâche reste « reconnaître les mêmes objets », $P(Y | X)$ ne change pas en principe — c'est un cas typique de covariate shift.
- Q1.2 Une campagne de recrutement médical qui change de tranche d'âge cible : $P(X | Y)$ (profil des patients avec/sans la pathologie) reste stable, mais la proportion de patients malades change selon la population recrutée.
- Q1.3 Plus les domaines sont différents, plus le meilleur compromis atteignable $\lambda_{\mathcal{H}}$ (risque joint minimal) risque d'être élevé lui-même : même la meilleure méthode ne peut pas faire mieux que cette limite structurelle.

Références

Ben-David, S., Blitzer, J., Crammer, K., Kulesza, A., Pereira, F., and Vaughan, J. W. (2010). A theory of learning from different domains. *Machine Learning*, 79(1-2) :151–175.

- Bickel, S., Brückner, M., and Scheffer, T. (2007). Discriminative learning for differing training and test distributions. In *Proceedings of the 24th ICML*, pages 81–88.
- Courty, N., Flamary, R., Habrard, A., and Rakotomamonjy, A. (2017). Joint distribution optimal transportation for domain adaptation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 30.
- Ganin, Y. and Lempitsky, V. (2015). Unsupervised domain adaptation by backpropagation. In *Proceedings of the 32nd ICML*, pages 1180–1189.
- Germain, P., Habrard, A., Laviolette, F., and Morvant, E. (2013). A PAC-Bayesian approach for domain adaptation with specialization to linear classifiers. In *Proceedings of the 30th ICML*, pages 738–746.
- Germain, P., Habrard, A., Laviolette, F., and Morvant, E. (2016). A new PAC-Bayesian perspective on domain adaptation. In *Proceedings of the 33rd ICML*, pages 859–868.
- Huang, J., Gretton, A., Borgwardt, K., Schölkopf, B., and Smola, A. J. (2007). Correcting sample selection bias by unlabeled data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 19, pages 601–608.
- Long, M., Cao, Y., Wang, J., and Jordan, M. (2015). Learning transferable features with deep adaptation networks. In *Proceedings of the 32nd ICML*, pages 97–105.
- Loog, M. (2012). Nearest neighbor-based importance weighting. In *IEEE International Workshop on Machine Learning for Signal Processing*, pages 1–6.
- Sugiyama, M., Nakajima, S., Kashima, H., Büna, P. v., and Kawanabe, M. (2007). Direct importance estimation with model selection and its application to covariate shift adaptation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 20, pages 1433–1440.
- Sun, B. and Saenko, K. (2016). Deep CORAL : correlation alignment for deep domain adaptation. In *ECCV Workshops*, pages 443–450.
- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. (2021). Tent : fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations (ICLR)*.
- Zhang, J., Ding, Z., Li, W., and Ogunbona, P. (2018). Importance weighted adversarial nets for partial domain adaptation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8156–8164.
- Zhao, H., des Combes, R. T., Zhang, K., and Gordon, G. (2019). On learning invariant representations for domain adaptation. In *Proceedings of the 36th ICML*, pages 7523–7532.
- Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H., and He, Q. (2021). A comprehensive survey on transfer learning. *Proceedings of the IEEE*, 109(1) :43–76.